



Sistema de gestión de kioscos en SQL Server Management Studio

Tópicos de Bases de Datos



Integrantes Grupo V:
Barvagelata, Julián Mariano
González De Rose, Franco Ezequiel
Gotte, Joaquín Ezequiel
Miranda Quisbert, Brian Alex

Parte 1- Explicación de Sistema de Kioskos

Se solicita un sistema de gestión de Kioskos. Se necesita registrar los empleados, detallando DNI, tipo documento, nombre, apellido, dirección, nacionalidad, edad, fecha nacimiento y el estado de empleado (si se encuentra activo o dado de baja). Se identificará el empleado gracias a su DNI y tipo de documento.

Un empleado puede supervisar a muchos empleados o a ninguno. y nos interesa registrar estas relaciones.

La única caja del turno debe ser atendida por un único empleado. Un empleado sólo puede manejar una única caja a la vez, pero no todos los empleados trabajan en la caja. Del turno de caja se debe registrar fecha inicio, fecha fin.

Se identifica el turno de caja mediante un id de turno más la identificación del empleado. ya que el id de turno se repite entre los empleados, pero no se repite en la gestión del mismo empleado. Además, del turno de caja se calculará el saldo inicial y el saldo de cierre.

Un turno de caja registra muchas o ninguna venta. Cada venta es registrada por un único turno de caja.

Respecto a las ventas, se almacena el modo de pago, se calcula el total de la venta, y se guarda la hora de la venta. Las ventas se identifican con un id de venta único.

Cada venta tiene asociado 1 o muchos detalles de venta. Pero cada detalle de venta está asociada a una única venta. Cada detalle de venta, además, cuenta con un nro_detalle. Este nro puede repetirse en otra venta, pero no en la misma venta. Además, el detalle de venta contiene la cantidad del producto, el precio unitario, y el cálculo del subtotal.

Cada detalle de venta está asociada a un único producto.

Respecto a los productos, se discriminan como bebidas y comestibles. Entre los comestibles, nos interesa almacenar su fecha de vencimiento. Para las bebidas, el grado de alcohol. Para todos los productos en general, se necesita registrar el detalle del producto y marca. Los productos se identificarán por un id único.

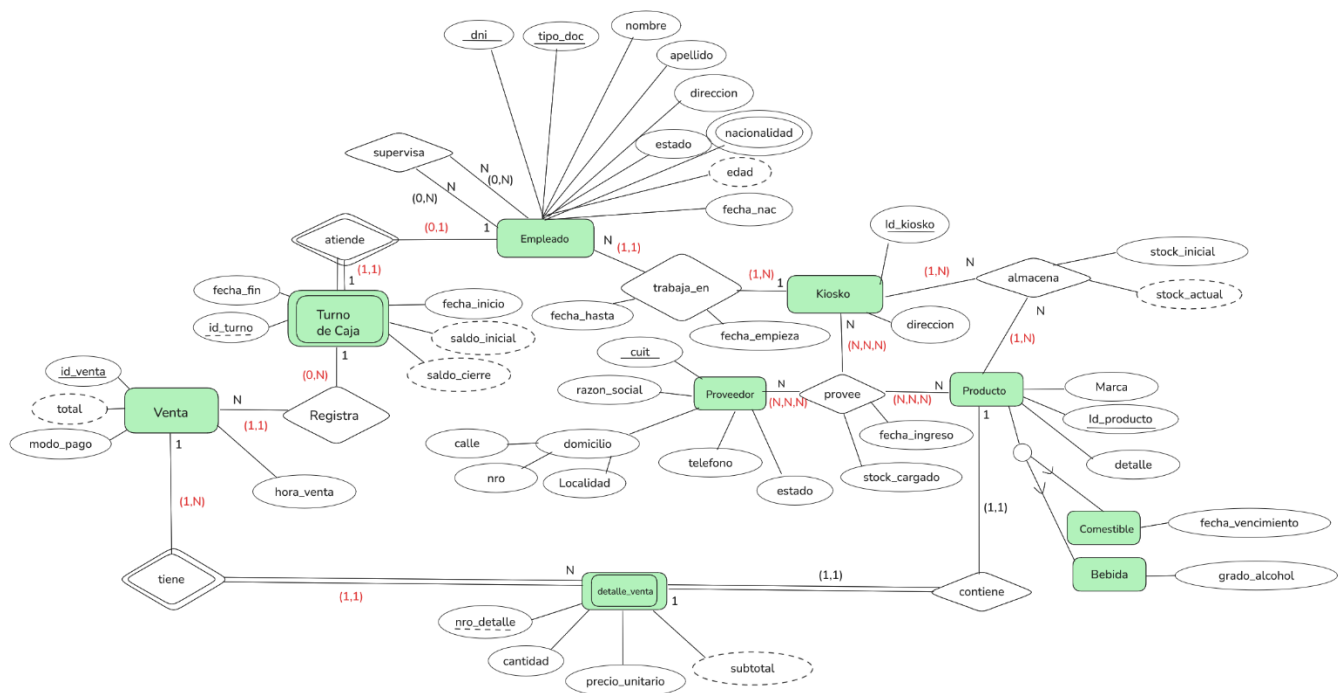
Un empleado solo puede trabajar en un mismo kiosko al mismo tiempo. En un kiosko puede trabajar uno o varios empleados. En la relación de este empleado con el kiosko se desea almacenar fecha de inicio y hasta.

Respecto a los kioskos, nos interesa registrar su dirección. Se identifican por un id único de kiosko. Cada kiosko almacena uno o varios productos, y cada producto es almacenado por uno o varios kioskos. Se desea almacenar el stock inicial y calcular el stock actual de cada producto en cada kiosko.

Respecto a los proveedores estos proveen diversos productos a diversos kioskos. En cada ingreso de stock almacenamos el stock cargado y la fecha de ingreso.

De los proveedores deseamos almacenar su razón social, su teléfono, su estado (si se encuentra activo o inactivo), domicilio (calle, nro, localidad). Se identifican por su cuit.

DER:



Herramienta Utilizada: <https://excalidraw.com/>

Modelo Relacional

EMPLEADO (dni (PK), tipo_doc (PK), nombre, apellido, direccion, estado, fecha_nac, id_kiosko (FK->kiosko.id_kiosko), fecha_hasta, fecha_empezar)

PK Compuesta -> (dni, tipo_doc)

#Nota: edad es calculado

NACIONALIDAD (dni (FK->empleado.dni), tipo_doc (FK->empleado.tipo_doc), nacionalidad)

PK Compuesta -> (dni, tipo_doc, nacionalidad)

#NACIONALIDAD es multivaluada (doble circulo), y uso clave primaria compuesta para evitar duplicados de DNI y tipo_doc.

SUPERVISA (dni (FK->empleado.dni), tipo_doc (FK->empleado.tipo_doc), dni_supervisado (FK->empleado.dni), tipo_doc_supervisado (FK->empleado.tipo_doc))

PK Compuesta -> (dni, tipo_doc, dni_supervisado, tipo_doc_supervisado)

TURNO_CAJA (id_turno, dni (FK->empleado.dni), tipo_doc (FK->empleado.tipo_doc), fecha_inicio, fecha_fin)

PK Compuesta -> (id_turno, dni, tipo_doc)

#Nota: saldo inicial y saldo_cierre son calculados.

VENTA (id_venta (PK), modo_pago, hora_venta, id_turno (FK->turno_caja.id_turno), dni (FK->turno_caja.dni), tipo_doc (FK->turno_caja.tipo_doc))

#Nota: el total es calculado.

DETALLE_VENTA (nro_detalle,cantidad,precio_unitario,id_venta (FK->venta.id_venta), id_producto (FK->producto.id_producto)

PK Compuesta ->(nro_detalle,id_venta)

#Nota: Subtotal es calculado

PRODUCTO (id_producto (PK), detalle, marca, tipo_producto, fecha_vencimiento, grado_alcohol)

#Para la generalización con Producto como superclase, y Comestible y Bebida como subclases se considera agregar columnas a la tabla Producto y adicionar el atributo tipo (tipo_producto).

KIOSKO (id_kiosko (PK), direccion)

ALMACENA (id_producto(FK->Producto.id_producto),id_kiosko(FK->kiosko.id_kiosko), stock_inicial)

#Nota: Stock_actual es calculado.

PROVEEDOR (cuit (PK), razon_social, calle, nro, localidad, telefono, estado)

#Para el caso del atributo compuesto Domicilio, se opta por eliminarlo y considerar todos sus componentes como atributos simples#

PROVEE (cuit (FK->Proveedor.cuit), id_producto(FK->producto.id_producto), id_kiosko(FK->Kiosko.id_kiosko, fecha_ingreso, stock_cargado)

PK Compuesta ->(cuit,id_producto,id_kiosko)

Parte 2 – Creación de BD en SQL Server Management Studio

Se insertan fotos del SQL.

```

1  /* =====
2  TRABAJO PRACTICO - GRUPO 5 - UNIVERSIDAD NACIONAL DE LA MATANZA
3  MATERIA TÓPICOS DE BASE DE DATOS
4  Se valido que el modelo relacional implementado ya se encontraba en la 3FN asegurando la eliminacion de redundancias.*/
5
6  /* =====
7  1- Creación de La Base de Datos y Esquema
8  • Crear una base de datos (BD) para el proyecto.
9  • El nombre de la BD será GRUPOS
10 • Definir al menos tres esquemas diferentes de dbo para organizar los objetos
11 Se crea la base de datos si es que no existe. Se llamará GRUPOS
12 =====
13 IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = 'GRUPOS')
14 BEGIN
15     CREATE DATABASE GRUPOS
16     ON -- Se genera la base con el create database version extendida, indicando tamaño, crecimiento, y path del archivo mdf. Ademas indicamos el archivo para el logging.
17     (
18         NAME = GRUPOS,
19         FILENAME = 'C:\GRUPOS\GRUPOS.mdf',
20         SIZE = 300MB,
21         MAXSIZE=900MB,
22         FILEGROWTH = 150MB
23     )
24     LOG ON
25     (
26         NAME = GRUPOS_LOG,
27         FILENAME = 'C:\GRUPOS\GRUPOS_log.ldf',
28         SIZE = 150MB,
29         MAXSIZE= 450MB,
30         FILEGROWTH = 150MB
31     );
32 END
33 GO
34 --Indicamos la base de datos en contexto, la cual sera GRUPOS. Al utilizar USE, indicamos que las sentencias se ejecutaran sobre esta base
35 USE GRUPOS;
36 GO
37
38 --Indicamos la base de datos en contexto, la cual sera GRUPOS. Al utilizar USE, indicamos que las sentencias se ejecutaran sobre esta base
39 USE GRUPOS;
40 GO
41
42 --se crean 3 esquemas DBO para organizar los objetos, podría agregarse autorización por rol en c/esquema. Pero lo omitimos en este paso ya que no fue solicitado.
43 IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'empleados')
44     EXEC('CREATE SCHEMA empleados');
45 GO
46
47 IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'operaciones')
48     EXEC('CREATE SCHEMA operaciones');
49 GO
50
51 IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'productos')
52     EXEC('CREATE SCHEMA productos');
53 GO
54
55 /* =====
56 2. Creación de Tablas
57 • Crear todas las tablas del modelo relacional definido en la Parte I. (empleado,nacionalidad,supervisa,detalle_venta,turno_caja,venta,almacena,kiosko,producto,provee,proveedor)
58 • Definir claves primarias en cada tabla. - Claves primarias definidas.
59 • Definir claves foráneas que aseguren la integridad referencial - Claves foraneas definidas.
60 • Especificar tipos de datos apropiados para cada columna. Utilice algun IDENTITY. - Identity se utiliza en Kiosko. Los tipos de datos se encuentran definidos.
61
62 3. Definición de Restricciones
63 • Aplicar restricciones NOT NULL donde corresponda. - Aplicado
64 • Definir al menos una restricción UNIQUE. - Se aplica en direccion en Kiosko
65 • Definir al menos una restricción CHECK. - Se aplica en grado_alcohol en tabla producto
66 • Definir al menos una restricción DEFAULT. - Se aplica en estado en tabla Proveedor.
67 =====
68
69 CREATE TABLE productos KIOSKO (
70     id_kiosko INT PRIMARY KEY IDENTITY(1,1), -- Se define int para definir el id que va ir sumando de 1 en 1. Arrancando en 1. Es la PK de la tabla.
71     direccion VARCHAR(100) NOT NULL UNIQUE, -- Se define varchar ya que es texto que se va a ingresar, necesitamos que no sea nulo ya que un id por si solo no nos brinda info. Ademas el UNIQUE, asegura que sea unico en la columna.
72     balance_kiosko NUMERIC --Se agrega balance en c/kiosko para reducir pagos a proveedores y sumar ventas. Esto no estaba contemplado en el DER. Se actualizará en un futuro mediante Triggers.
73 );
74 GO
75
76 CREATE TABLE empleados EMPLEADO (
77     nro_doc NUMERIC(9) NOT NULL, -- Se define numerica ya que se admite solamente tipo_doc como "documento extranjero" o "dni". No se admiten pasaportes para el dominio de gestion de Kioskos.
78     tipo_doc VARCHAR(10) NOT NULL, -- Es un varchar que admite texto. Se valida el dominio y se decide que sea "documento extranjero" o "dni".
79     nombre VARCHAR(50) NOT NULL, -- nombre del empleado, se admiten hasta 50 caracteres. No se permiten nulos.
80     apellido VARCHAR(50) NOT NULL, -- Apellido del empleado, se admiten hasta 50 caracteres. No se permiten Nulos.
81     direccion VARCHAR(100), -- Direccion del empleado, se admiten hasta 100 caracteres. Se admiten Nulos.
82     estado BIT NOT NULL DEFAULT 1, -- ACTIVO=1 Se genera un valor que puede valer 0 y 1. El valor 1 es por defecto y corresponde a activo. 0=inactive.
83     fecha_nac DATE, -- Se define el atributo de fecha de nacimiento con tipo de dato fecha.
84     edad AS DATEDIFF(YEAR, fecha_nac, GETDATE()), -- Se define atributo calculado y se obtiene a partir de la fecha de nacimiento y la fecha de hoy. Solo en años corridos.
85     id_kiosko INT NOT NULL, -- Es una FK proveniente de la tabla de Kiosko. Identifica a que kiosko trabaja el empleado. Solo puede trabajar en 1 mismo kiosko. Se define NOT NULL ya que es importante que tengamos esta relacion.
86     fecha_hasta DATE, -- Igual comentario que fecha de nacimiento. Se completa automaticamente con Null hasta que se modifica el estado del empleado. Puede hacerse mediante un trigger cuando se realiza un update del estado.
87     fecha_emplea DATE NOT NULL, -- Igual comentario que fecha de nacimiento, se carga la fecha de alta cuando se realiza un insert de un empleado. Se necesita conocer el dato en el ingreso asi que se define el NOT NULL.
88     telefono VARCHAR(20), -- se agrega nuevo atributo no contemplado en el DER ni en modelo relacional ya que consideramos que es importante este valor.
89     PRIMARY KEY (nro_doc, tipo_doc), -- Se continua con lo definido en el modelo relacional. Se define las claves primarias.
90     FOREIGN KEY (id_kiosko) REFERENCES productos KIOSKO(id_kiosko) -- Se continua con lo definido en el modelo relacional. Se define las claves foraneas.
91 );
92 GO
93
94 CREATE TABLE empleados NACIONALIDAD (
95     nro_doc NUMERIC(9) NOT NULL, -- tipo y definicion explicados en la tabla empleado.
96     tipo_doc VARCHAR(10) NOT NULL, -- tipo y definicion explicados en la tabla empleado.
97     nacionalidad VARCHAR(30) NOT NULL, -- Es una FK proveniente de la tabla de Kiosko. Identifica a que kiosko trabaja el empleado. Solo puede trabajar en 1 mismo kiosko. Se define NOT NULL ya que es parte de la clave primaria. Se define VARCHAR de 30 digitos para contemplar los casos.
98     PRIMARY KEY (nro_doc, tipo_doc, nacionalidad), -- Se define la clave primaria segun lo definido en el modelo relacional.
99     FOREIGN KEY (nro_doc, tipo_doc) REFERENCES empleados EMPLEADO(nro_doc, tipo_doc) -- Se define la claves foraneas segun lo definido en el modelo relacional.
100 );
101 GO
102
103 CREATE TABLE empleados SUPERVISA (
104     nro_doc NUMERIC(9) NOT NULL, -- Tipo y definicion explicados en la tabla empleado.
105     tipo_doc VARCHAR(10) NOT NULL, -- Tipo y definicion explicados en la tabla empleado.
106     nro_doc_superviso NUMERIC(9) NOT NULL, -- Se define igual que nro_doc
107     tipo_doc_superviso VARCHAR(10) NOT NULL, -- Se define igual que tipo_doc
108     PRIMARY KEY (nro_doc, tipo_doc, nro_doc_superviso, tipo_doc_superviso), -- Se define la clave primaria segun lo definido en el modelo relacional
109     FOREIGN KEY (nro_doc, tipo_doc) REFERENCES empleados EMPLEADO(nro_doc, tipo_doc), -- Se define las clave foraneas segun lo definido
110     FOREIGN KEY (nro_doc_superviso, tipo_doc_superviso) REFERENCES empleados EMPLEADO(nro_doc, tipo_doc) -- Se define las claves foraneas segun lo definido
111 );
112 GO
113
114
115
116
117
118
119

```

```

108 GO
109
110 CREATE TABLE operaciones.TURNO_CAJA (
111     id_turno INT NOT NULL, -- Not null ya que es parte de la primary key. No se agrega el identity ya el valor del turno se reinicia por empleado, se puede repetir por empleado.
112     nro_doc NUMERIC(9) NOT NULL, -- Tipo y definicion explicados en la tabla empleado
113     tipo_doc VARCHAR(10) NOT NULL, -- Tipo y definicion explicados en la tabla empleado
114     fecha_inicio DATETIME DEFAULT GETDATE(), -- Se toma la fecha por defecto de cuando se realiza un insert
115     fecha_fin DATETIME, -- Se carga fecha fin de cuando cierra el turno de caja.
116     saldo_inicial NUMERIC, -- Se agrega saldo inicial como NUMERIC. Se va a utilizar mediante triggers en un futuro.
117     saldo_cierre NUMERIC, -- Se agrega saldo cierre como numeric. Se va a utilizar mediante triggers en un futuro.
118     PRIMARY KEY (id_turno, nro_doc, tipo_doc), -- Se define la PK segun lo definido en el modelo relacional
119     FOREIGN KEY (nro_doc, tipo_doc) REFERENCES empleados.EMPLEADO(nro_doc, tipo_doc) -- Se define la FK segun lo definido en el modelo relacional.
120 );
121 GO
122
123 CREATE TABLE operaciones.VENTA (
124     id_venta INT PRIMARY KEY NOT NULL IDENTITY (1,1), -- Se agrega el NOT NULL. En este caso es innecesario ya que es parte de la PK. Se agrega el identity ya que el valor es unico entre todas las ventas.
125     modo_pago VARCHAR(20), -- Se define modo pago como varchar para soportar distintos medios de pago. Ejm: "Efectivo", "Tarjeta Debito"
126     calc_venta NUMERIC not null DEFAULT 0 CHECK (calc_venta >= 0), -- Se define el atributo para calcular el total de las ventas, como defecto lo seteamos en 0 ya que una venta arranca en 0.
127     estado VARCHAR(10) DEFAULT 'ABIERTA', -- se agrega este dato para despues utilizarlo para actualizar el balance del kiosko. Se utilizara como trigger luego de un update en este valor.
128     --El valor se actualizara mediante triggers sumando los subtotales de los detalles de las ventas.
129     hora_venta DATETIME DEFAULT GETDATE(), -- Se define la hora de la venta como fecha por defecto cuando se inserta.
130     id_turno INT NOT NULL, -- Se define en la tabla turno caja
131     nro_doc NUMERIC(9) NOT NULL, -- Se define en la tabla empleado
132     estado VARCHAR(10) DEFAULT 'ABIERTA', -- se agrega este dato para despues utilizarlo para actualizar el balance del kiosko. Se utilizara como trigger luego de un update en este valor.
133     --De lo contrario, necesitaría ir actualizando el balance del kiosko en cada detalle de venta lo cual no resulta prolijo.
134     tipo_doc VARCHAR(10) NOT NULL, -- Se define en la tabla empleado
135     FOREIGN KEY (id_turno, nro_doc, tipo_doc) REFERENCES operaciones.TURNO_CAJA(id_turno, nro_doc, tipo_doc) -- Se define la FK segun lo definido en el MR
136 );
137 GO
138
139 CREATE TABLE productos.PRODUCTO (
140     id_producto INT PRIMARY KEY NOT NULL IDENTITY(1,1), -- Se define como int y PK. Se define el identity para que vaya creciendo de 1 en 1, arrancando en 1.
141     detalle NVARCHAR(150), -- Se define el detalle del producto con un limite de 150 caracteres. Se define como NVARCHAR para soportar caracteres unicode. Para soportar productos con nombres internacionales.
142     marca VARCHAR(50), -- Se define para soportar caracteres unicode hasta 50.
143     tipo_producto VARCHAR(20), -- Se define varchar20 para soportar caracteres unicode hasta 20 de tipo de productos.
144     fecha_vencimiento DATE, -- Se define como tipo fecha el vencimiento
145     grado_alcohol DECIMAL(4,2) CHECK (grado_alcohol >= 0)-- RECORDAR QUE SIEMPRE DEBE SER MAYOR A 0. NO EXISTE BEBIDA CON GRADO ALCOHOL NEGATIVO.
146 );
147 GO
148
149 CREATE TABLE operaciones.DETALLE_VENTA (
150     nro_detalle INT NOT NULL, -- Se define not null ya que es parte de la PK. No se define el IDENTITY ya que se resetea por cada venta.
151     cantidad SMALLINT CHECK (cantidad > 0) NOT NULL, -- Se define la cantidad como smallint ya que ocupa menos espacio. Se añade un check ya que no puede ser negativo.
152     precio_unitario DECIMAL(16,2) NOT NULL, -- DDO CADA KIOSKO ESTIPULA PRECIO - CABA Y PCIA DIFERENTES PRECIOS. No lo tomamos de productos. Se ingresa en c/detalle de venta. Sera un numero de max de 10 digitos.
153     --8 para la parte entera y 2 decimales.
154     id_venta INT NOT NULL, -- Se define en la tabla venta
155     id_producto INT NOT NULL, -- Se define en la tabla producto
156     subtotal AS (cantidad * precio_unitario) -- columna subtotal es calculada de tipo DECIMAL. Es un valor calculado en la misma tabla. Se puede definir en el create.
157     PRIMARY KEY (id_venta, nro_detalle), -- Se define la PK segun lo definido en el MR
158     FOREIGN KEY (id_venta) REFERENCES operaciones.VENTA(id_venta), -- Se define la FK segun lo definido en el MR
159     FOREIGN KEY (id_producto) REFERENCES productos.PRODUCTO(id_producto) -- Se define la FK segun lo definido en el MR
160 );
161 GO
162
163 CREATE TABLE productos.ALMACENA (
164     id_producto INT NOT NULL, -- Se define en la tabla producto
165     id_kiosko INT NOT NULL, -- Se define en la tabla kiosko
166     stock_actual SMALLINT not null DEFAULT 0 CHECK (stock_actual >= 0), -- Se calcula como SMALLINT para stocks mayor a 255 de un producto. Se agrega un check ya que no puede ser negativo y no nulo. Por defecto los seteamos en 0
167     CONSTRAINT PK_ALMACENA PRIMARY KEY NONCLUSTERED (id_kiosko, id_producto), -- Se define Non cluster la PK ya que queremos que la tabla se ordene por el ID de kiosko. Se define el indice mas adelante.
168     FOREIGN KEY (id_producto) REFERENCES productos.PRODUCTO(id_producto), -- Se define la FK segun lo definido en el MR
169     FOREIGN KEY (id_kiosko) REFERENCES productos.KIOSKO(id_kiosko) -- Se define la FK segun lo definido en el MR
170 );
171 GO
172
173 CREATE TABLE productos.PROVEEDOR (
174     cuit CHAR(11) PRIMARY KEY NOT NULL, -- Se define el cuit como PK segun el MR. Se define como char de 11 caracteres. Se debe cargar sin espacios
175     razon_social VARCHAR(100) NOT NULL, --Se agrega campo de razon social.
176     calle VARCHAR(50), -- se agrega campo de calle para el proveedor
177     nro_calle SMALLINT, -- se agrega nro de calle como smallint para ahorrar espacio.
178     localidad VARCHAR(50), -- Se agrega localidad como varchar de max 50 caracteres
179     telefono VARCHAR(20), -- Se agrega telefono del proveedor.
180     estado BIT NOT NULL DEFAULT 1 --ACTIVO=1 -- Se agrega el estado de un proveedor si esta activo o no. similar a lo explicado en tabla empleado.
181 );
182 GO
183
184 CREATE TABLE productos.PROVEE (
185     cuit CHAR(11) NOT NULL, -- Explicado en tabla proveedor
186     id_producto INT NOT NULL, -- Explicado en tabla producto
187     id_kiosko INT NOT NULL, -- Explicado en tabla kiosko
188     fecha_ingreso DATE NOT NULL, -- Se define fecha de ingreso como tipo fecha y not NULL ya que es parte de la PK
189     stock_cargado SMALLINT CHECK (stock_cargado > 0), -- Se agrega el stock cargado como proveedor como smallint y se agrega un check de mayor a 0
190     pago_proveedor NUMERIC NOT NULL, -- se agrega el pago al proveedor. No contemplado en el DER. Se agrega not null para no cargar ninguna fila sin un valor.
191     CONSTRAINT PK_PROVEE PRIMARY KEY NONCLUSTERED (cuit, id_producto, id_kiosko, fecha_ingreso), --Correccion se agrega fecha_ingreso como primary key. Esto es una correccion del MR.
192     --Un error del modelo realizado. No contemplaba otra carga ( fue validado con lote de pruebas)
193     FOREIGN KEY (cuit) REFERENCES productos.PROVEEDOR(cuit), -- Se define la FK segun mr
194     FOREIGN KEY (id_producto) REFERENCES productos.PRODUCTO(id_producto), -- Se define FK segun MR
195     FOREIGN KEY (id_kiosko) REFERENCES productos.KIOSKO(id_kiosko) -- Se define segun MR.
196 );
197 GO
198

```

```

198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311

```

/*****
 4- Índices
 • Crear al menos un índice clustered y uno nonclustered por tabla.
 • Incorporar un índice de cobertura (covering index) para consultas frecuentes.
 *****/
 -Los índices clustered se aplican sobre claves primarias para optimizar búsquedas por identidad. Los nonclustered permiten búsquedas por atributos secundarios como nombre o kiosko.
 pensar que es un atajo hacia otras tablas de atributo (las que figuran luego de ON en el código).
 Creo índices no agrupados (nonclustered) para mejorar rendimiento en búsquedas sin variar el orden de los datos
 Índice no agrupados son como "índices de libro": apuntan a dónde están los datos, pero no los reordenan.
 -Los ind cobertura son ind NO AGRUPADOS que tienen todas las columnas necesarias para resolver una consulta sin tener que recurrir a la tabla base; el índice presenta todas las columnas
 que la consulta necesita tanto para filtrar como WHERE, como para proyectar con SELECT.
 El índice de cobertura mejora el rendimiento de consultas frecuentes sobre ventas.
 -Una tabla en SQL Server solo puede tener un único índice clustered, ya que define el orden físico de los datos en la tabla.
 -CLAVE: SON LOS ATRIBUTOS QUE SON PK o PKFK. Si es solo PK NO es clave. Se caracterizan visualmente por una llave color negro en el panel izquierdo.
 Si quiero agregar a nonclustered debere usar INCLUDE.
 *****/
 -- LOS PK SE GENERAN AUTOMATICAMENTE COMO CLUSTER SI NO DEFINIMOS UNO PREVIAMENTE. EN PROVIDE SI PODEMOS DEFINIR YA QUE DEFINIMOS LA PK COMO NONCLUSTERED:
 CREATE CLUSTERED INDEX IX_PROVEE_FECHA
 ON productos.PROVEE(fecha_ingreso); -- Ordenamos físicamente la tabla por fecha de ingreso de la tabla provee. Nos permite listar los ingresos de los proveedores
 CREATE CLUSTERED INDEX IX_ALMACENA_KIOSKO
 ON productos.almacena(id_kiosko); -- Ordenamos la tabla físicamente por id de KIOSKO. Ordenamos la tabla almacena que tiene producto kiosko y stock.
 CREATE NONCLUSTERED INDEX IX_DETALLE_VENTA_CANT ON operaciones.DETALLE_VENTA(id_venta,id_producto,cantidad); -- Índice para mejorar consultas que filtren por id_venta,id_producto y queramos obtener la cantidad de ventas.
 GO
 CREATE NONCLUSTERED INDEX IX_TURNO_CAJA_NRO ON operaciones.TURNO_CAJA(nro_doc); -- Índice para mejorar consultas por nro_doc para obtencion de todos los turnos asociados.
 GO
 CREATE NONCLUSTERED INDEX IX_VENTA_HORA ON operaciones.VENTA(id_venta) --Cubre consultas por id_venta y hora_venta sin acceder a tabla. Es un índice de cobertura.
 INCLUDE (hora_venta);
 GO
 CREATE NONCLUSTERED INDEX IX_VENTA_MODO_PAGO ON operaciones.VENTA(modo_pago) -- Cubre consultas cuando busco por modo de pago en el where.
 INCLUDE (id_venta, hora_venta);
 GO
 CREATE NONCLUSTERED INDEX IX_EMPLEADO ON empleados.EMPLEADO(nro_doc) --búsquedas por nro_doc y mostrar datos personales.
 INCLUDE (nombre, apellido,id_kiosko, telefono);
 GO
 CREATE NONCLUSTERED INDEX IX_NACIONALIDAD ON empleados.NACIONALIDAD(nro_doc,nacionalidad); -- busqueda por nacionalidad y nro_DOC
 GO
 CREATE NONCLUSTERED INDEX IX_SUPERVISA ON empleados.SUPERVISA(nro_doc, nro_doc_supervisado); -- Busqueda por nro_doc y nro_Doc supervisado
 GO
 CREATE NONCLUSTERED INDEX IX_ALMACENA ON productos.ALMACENA(id_kiosko) -- Busqueda por id_kiosko
 INCLUDE (id_producto);
 GO
 CREATE NONCLUSTERED INDEX IX_KIOSKO_DIRECCION ON productos.KIOSKO(id_kiosko) --muestra dirección al buscar por id_kiosko.
 INCLUDE (direccion);
 GO
 CREATE NONCLUSTERED INDEX IX_PRODUCTO ON productos.PRODUCTO(id_producto) -- Cubre consultas por id_producto y atributos incluidos.
 INCLUDE (marca,detalle,fecha_vencimiento, grado_alcohol);
 GO
 CREATE NONCLUSTERED INDEX IX_PROVEE ON productos.PROVEE(cuit,id_producto,id_kiosko); -- busqueda por cuit, id_Producto e id_kiosko
 GO
 CREATE NONCLUSTERED INDEX IX_PROVEEDOR_TELEFONO ON productos.PROVEEDOR(cuit) --Cubre consultas por cuit y teléfono sin acceder a tabla.
 INCLUDE (telefono);
 GO
 /*****
 5-Modificación de Tablas (ALTER)
 •Agregar una nueva columna a una tabla. (se agregará columna origen a producto y telefono a kiosko)
 • Modificar el tipo de dato de una columna existente. (se modifica el tipo de dato a detalle de la tabla producto)
 • Eliminar una columna de una tabla (que no sea clave primaria o foránea). (se elimina la columna origen creada en el primer punto)
 *****/
 ALTER TABLE nos permite alterar/modificar la estructura de una tabla existente (sumar o eliminar columnas, cambiar tipo dato de col, o sumar o eliminar restricciones)
 *****/
 ALTER TABLE productos.PRODUCTO ADD origen VARCHAR(50); --se agrega la columna origen a la tabla.
 GO
 ALTER TABLE empleados.EMPLEADO -- se agrega columna antigüedad en años
 ADD antigüedad_año AS DATEDIFF(YEAR, fecha_empez, GETDATE());
 GO
 ALTER TABLE productos.KIOSKO ADD cel VARCHAR(20);--se agrega numero telefonico del kiosko
 GO
 ALTER TABLE productos.PRODUCTO ALTER COLUMN detalle NVARCHAR(200); --Se cambia detalle hasta 200 caracteres
 GO
 ALTER TABLE productos.PRODUCTO DROP COLUMN origen;--se elimina la columna origen de la tabla producto
 GO
 .
 /*****
 6-Eliminación de Objetos
 • Eliminar al menos un objeto con DROP.
 • Vaciar los datos de una tabla con TRUNCATE.
 DROP elimina objetos no necesarios, mientras que TRUNCATE permite reiniciar datos sin afectar la estructura.
 *****/
 -- Eliminar índice de nacionalidad (recordar refrescar el explorador de objetos, ya que tarda unos segundos)
 DROP INDEX IX_NACIONALIDAD ON empleados.NACIONALIDAD;
 GO
 -- Vaciar tabla de detalle para simular reinicio de operaciones
 TRUNCATE TABLE operaciones.DETALLE_VENTA;
 GO
 --Se genera nuevamente el indice en nacionalidad
 CREATE NONCLUSTERED INDEX IX_NACIONALIDAD ON empleados.NACIONALIDAD(nro_doc,nacionalidad); -- busqueda por nacionalidad y nro_DOC
 GO

```

311
312  /*=====
313  7-Estructuras Temporales
314  Crear una tabla temporal local.
315  • Crear una tabla temporal global.
316  • Crear una variable de tabla.
317  Trate de proponer su uso potencial en el contexto de su sistema, justificando su utilidad.
318  Las estructuras temporales permiten manejar datos intermedios sin afectar el modelo persistente.
319  =====*/
320
321  /*Al ser local, se encuentra solo disponible en la sesion actual ya que al cerrar la sesion
322  se eliminará automaticamente.
323  El uso que tendra en el sistema es para almacenar resultados intermedios.
324  En el sistema de kiosko servirá para filtrar productos especificos de un kiosko para generar un reporte temporal.
325  */
326
327  DROP TABLE IF EXISTS #Temp_Productos;
328  GO
329  CREATE TABLE #Temp_Productos (
330      id_producto INT,
331      nombre VARCHAR(50)
332  );
333  GO
334
335  /* Se encuentra disponible para todas las sesiones, mientras exista alguna sesion que la este usando.
336  Podrá mantener un resumen global del stock de todos los kioskos sin tener que modificar la tabla de almacena.
337  Se puede consultar por varios usuarios.
338  */
339
340  DROP TABLE IF EXISTS ##Global_Stock;
341  GO
342  CREATE TABLE ##Global_Stock (
343      id_kiosko INT,
344      stock_total INT
345  );
346  GO
347
348  /*
349  Al ser una variable de tabla solo se utilizará en el mismo procedimiento donde se declara.
350  La siguiente definicion guarda temporalmente los turnos asignados a empleados.
351  */
352
353  DECLARE @TablaTurnos TABLE (
354      id_turno INT,
355      fecha_inicio DATETIME
356  );
357  GO
358
359

```

Parte 3 – Consultas, Funciones, UDF, Ventanas, Expresiones comunes de Tabla y Vistas

Se insertan fotos del SQL, esquemas y tablas.

```

/* TP3 - GRUPO 5 */

USE GRUPO5;
GO

/*
1. Inserción de Datos (INSERT)
Se desarrollarán scripts de inserción para poblar las tablas principales del modelo relacional. Los datos serán
representativos y permitirán realizar consultas significativas.
• Se incluirán al menos 5 registros por tabla.
• Se respetarán las claves foráneas y las restricciones definidas en la Parte II. */

-- =====
-- KIOSKO
-- =====
INSERT INTO productos.KIOSKO (direccion, cel)
VALUES
('Av. Libertador 123', '1111-1111'),
('Calle Falsa 456', '2222-2222'),
('Av Corrientes 456', '3333-3333'),
('Belgrano 456', '4444-4444'),
('Av. San Martin 789', '5555-5555');
GO

-- =====
-- EMPLEADO
-- =====
INSERT INTO empleados.EMPLEADO
(nro_doc, tipo_doc, nombre, apellido, direccion, fecha_nac, id_kiosko, fecha_empieza, fecha_hasta, telefono)
VALUES
(203040506, 'DNI', 'Juan', 'Perez', 'Calle A 123', '1985-05-12', 1, '2024-01-01', NULL, '1234-5678'),
(203040507, 'DNI', 'Ana', 'Gomez', 'Calle B 456', '1990-08-20', 1, '2024-02-01', NULL, '2345-6789'),
(203040508, 'DNI', 'Luis', 'Martinez', 'Calle C 789', '1988-03-15', 1, '2024-03-01', NULL, '3456-7890'),
(203040509, 'DNI', 'Franco', 'Gonzalez', 'Calle Z 985', '1998-10-24', 2, '2025-07-24', NULL, '3067-0851'),
(203040510, 'DNI', 'Ezequiel', 'Garcia', 'Calle Z 986', '1997-10-24', 2, '2024-07-30', NULL, '3067-0852'),
(203040511, 'DNI', 'Leandro', 'Gimenez', 'Calle M 486', '1996-10-24', 3, '2025-05-17', NULL, '3067-0853'),
(203040512, 'DNI', 'Brian', 'Gimenez', 'Calle W 6', '1996-10-24', 3, '2025-05-17', NULL, '3067-0853'),
(203040513, 'DNI', 'Diego', 'Sanchez', 'Calle X 450', '1995-10-24', 3, '2025-01-07', NULL, '3067-0854');
GO

-- =====
-- NACIONALIDAD
-- =====
INSERT INTO empleados.NACIONALIDAD (nro_doc, tipo_doc, nacionalidad)
VALUES
(203040506, 'DNI', 'Argentina'),
(203040506, 'DNI', 'Española'),
(203040507, 'DNI', 'Argentina'),
(203040508, 'DNI', 'Argentina'),
(203040509, 'DNI', 'Argentina'),
(203040510, 'DNI', 'Italiana'),
(203040510, 'DNI', 'Argentina'),
(203040511, 'DNI', 'Argentina'),
(203040511, 'DNI', 'Paraguaya'),
(203040512, 'DNI', 'Argentina');
GO

```

```
-- =====
-- SUPERVISA
-- =====
INSERT INTO empleados.SUPERVISA (nro_doc, tipo_doc, nro_doc_supervisado, tipo_doc_supervisado)
VALUES
(203040506, 'DNI', 203040507, 'DNI'),
(203040506, 'DNI', 203040508, 'DNI'),
(203040509, 'DNI', 203040510, 'DNI'),
(203040511, 'DNI', 203040512, 'DNI'),
(203040511, 'DNI', 203040513, 'DNI');
GO

-- =====
-- PRODUCTO
-- =====
INSERT INTO productos.PRODUCTO (detalle, marca, tipo_producto, fecha_vencimiento, grado_alcohol)
VALUES
('Cerveza rubia 500ml', 'Patagonia', 'Bebida', '2025-12-31', 5.0),
('Vino tinto 750ml', 'Norton', 'Bebida', '2026-06-30', 12.0),
('Gaseosa cola 2L', 'Coca-Cola', 'Bebida', '2025-11-30', 0.0),
('Galletitas', 'Trio', 'Galletas', '2025-11-30', NULL),
('Papas Fritas', 'Pepsi co', 'Snacks', '2025-11-30', NULL),
('Dulce de Leche', 'La Serenisima', 'Lacteos', '2025-12-30', NULL);
GO

INSERT INTO productos.PRODUCTO (detalle, marca, tipo_producto, fecha_vencimiento, grado_alcohol)
VALUES
('Fideos', 'Vicente', 'Pastas', '2025-12-31', 0);

-- =====
-- ALMACENA
-- =====
INSERT INTO productos.ALMACENA (id_producto, id_kiosko)
VALUES
(1, 1),
(2, 1),
(3, 1),
(4, 1),
(5, 1),
(6, 1),
(1, 2),
(2, 2),
(3, 2),
(4, 2),
(5, 2),
(6, 2),
(1, 3),
(2, 3),
(3, 3),
(4, 3),
(5, 3),
(6, 3);
GO
```

```

-- =====
-- PROVEEDOR
-- =====
INSERT INTO productos.PROVEEDOR (cuit, razon_social, calle, nro_calle, localidad, telefono)
VALUES
('20304050601', 'Distribuidora A', 'Calle 1', 100, 'Ciudad A', '1111-1111'),
('20304050602', 'Distribuidora B', 'Calle 2', 200, 'Ciudad B', '2222-2222'),
('20304050603', 'Distribuidora C', 'Calle 3', 300, 'Ciudad A', '3333-3333'),
('20304050604', 'Distribuidora D', 'Calle 4', 400, 'Ciudad C', '4444-4444'),
('20304050605', 'Distribuidora E', 'Calle 5', 500, 'Ciudad D', '5555-5555'),
('20304050606', 'Distribuidora F', 'Calle 6', 600, 'Ciudad F', '6666-6666');
GO

-- =====
-- PROVEE
-- =====
INSERT INTO productos.PROVEE (cuit, id_producto, id_kiosko, fecha_ingreso, stock_cargado, pago_proveedor)
VALUES
('20304050601', 1, 1, '2025-08-01', 50, 100),
('20304050602', 2, 1, '2025-08-01', 30, 100),
('20304050603', 3, 1, '2025-08-01', 100, 100),
('20304050604', 4, 1, '2025-08-01', 40, 100),
('20304050605', 5, 1, '2025-08-01', 40, 100),
('20304050606', 6, 1, '2025-08-01', 10, 100),
('20304050601', 1, 2, '2025-08-01', 10, 100),
('20304050602', 2, 2, '2025-08-01', 20, 100),
('20304050603', 3, 2, '2025-08-01', 30, 100),
('20304050604', 4, 2, '2025-08-01', 40, 100),
('20304050605', 5, 2, '2025-08-01', 50, 100),
('20304050606', 6, 2, '2025-08-01', 60, 100),
('20304050601', 1, 3, '2025-08-01', 20, 100),
('20304050602', 2, 3, '2025-08-01', 30, 100),
('20304050603', 3, 3, '2025-08-01', 40, 100),
('20304050604', 4, 3, '2025-08-01', 50, 100),
('20304050605', 5, 3, '2025-08-01', 60, 100),
('20304050606', 6, 3, '2025-08-01', 70, 100);
GO

-- =====
-- TURNOS
-- =====
--Validar como se valida el kiosko con la persona, como esta ahora, puede ser que en un mismo horario haya mas de una persona en la caja
INSERT INTO operaciones.TURNO_CAJA (id_turno, nro_doc, tipo_doc, fecha_inicio, fecha_fin)
VALUES
(1, 203040506, 'DNI', '2025-08-01 08:00', '2025-08-01 16:00'),
(1, 203040507, 'DNI', '2025-08-02 08:00', '2025-08-02 16:00'),
(1, 203040508, 'DNI', '2025-08-03 08:00', '2025-08-03 16:00'),
(1, 203040509, 'DNI', '2025-08-01 08:00', '2025-08-01 16:00'),
(1, 203040510, 'DNI', '2025-08-02 08:00', '2025-08-02 16:00'),
(1, 203040511, 'DNI', '2025-08-01 08:00', '2025-08-01 16:00'),
(1, 203040512, 'DNI', '2025-08-02 08:00', '2025-08-02 16:00'),
(1, 203040513, 'DNI', '2025-08-03 08:00', '2025-08-03 16:00');
GO

```

```

-- =====
-- VENTA
-- =====
INSERT INTO operaciones.VENTA (modo_pago, hora_venta, id_turno, nro_doc, tipo_doc)
VALUES
('Efectivo', '2025-08-01 08:30', 1, 203040506, 'DNI'),
('Efectivo', '2025-08-01 10:30', 1, 203040506, 'DNI'),
('Efectivo', '2025-08-01 11:30', 1, 203040506, 'DNI'),
('Efectivo', '2025-08-01 12:30', 1, 203040506, 'DNI'),
('Tarjeta Debito', '2025-08-01 14:30', 1, 203040506, 'DNI');
GO

```

```

-- =====
-- DETALLE_VENTA
-- =====
INSERT INTO operaciones.DETALLE_VENTA (nro_detalle,cantidad, precio_unitario, id_venta, id_producto)
VALUES
(1,2, 250.00, 1, 1),
(2,2, 800.00, 1, 2),
(3,2, 800.00, 1, 2),
(4,2, 800.00, 1, 2),
(1,3, 150.00, 2, 3),
(1,3, 150.00, 3, 3),
(1,3, 150.00, 4, 3),
(1,3, 150.00, 5, 3);
GO

```

```

203
204  /* Consultas con JOIN y Subconsultas
205  2) Se presentarán consultas que involucren:
206  • INNER JOIN, LEFT JOIN, y FULL JOIN entre entidades relacionadas.
207  • Subconsultas (Operadores IN, EXISTS, ANY y ALL).
208  • Cada consulta estará acompañada de su enunciado descriptivo. Esto es para todos los puntos.
209  • De ser posible, la imagen correspondiente a los datos devueltos. Esto es para todos los puntos.*/
210
211  --Inner Join  donde nos permite identificar el lugar de trabajo de cada empleado.
212
213  SELECT e.nombre, e.apellido, k.direccion
214  FROM empleados.EMPLEADO e
215  INNER JOIN productos.KIOSKO k ON e.id_kiosko = k.id_kiosko;

```

100 % 13 0

	nombre	apellido	direccion
1	Juan	Perez	Av. Libertador 123
2	Ana	Gomez	Av. Libertador 123
3	Luis	Martinez	Av. Libertador 123
4	Franco	Gonzalez	Calle Falsa 456
5	Ezequiel	Garcia	Calle Falsa 456
6	Leandro	Gimenez	Av Corrientes 456
7	Brian	Gimenez	Av Corrientes 456
8	Diego	Sanchez	Av Corrientes 456

```

216
217  --Left Join -- Donde nos permite visualizar si aún no tuvimos algun ingreso de algun producto
218  SELECT e.id_producto, e.detalle, k.fecha_ingreso
219  FROM productos.producto e
220  LEFT JOIN productos.PROVEE k ON e.id_producto = k.id_producto
221  WHERE k.fecha_ingreso is NULL
222
223  --Full Join - Se realiza Full join para obtener los productos que nunca se vendieron.
224
225  SELECT e.id_producto, e.detalle, k.fecha_ingreso

```

100 % 13 0

	id_producto	detalle	fecha_ingreso
1	7	Fideos	NULL

```

223 --Full Join - Se realiza Full join para obtener los productos que nunca se vendieron.
224
225 SELECT p.id_producto, p.detalle, dv.id_venta, dv.cantidad
226 FROM productos.PRODUCTO p
227 FULL JOIN operaciones.DETALLE_VENTA dv ON p.id_producto = dv.id_producto
228 WHERE dv.id_venta IS NULL
229
230 --Se define consulta para devolver los empleados que tuvieron alguna venta. Se utiliza "IN"
231
232 SELECT e.nombre, e.apellido, e.nro_doc, e.tipo_doc
233 FROM empleados.EMPLEADO e
234 WHERE e.nro_doc IN (
235 SELECT k.nro_doc
236 FROM operaciones.VENTA K );

```

100 % 13 0

Resultados Mensajes

	id_producto	detalle	id_venta	cantidad
1	4	Galletitas	NULL	NULL
2	5	Papas Fritas	NULL	NULL
3	6	Dulce de Leche	NULL	NULL
4	7	Fideos	NULL	NULL

```

230 --Se define consulta para devolver los empleados que tuvieron alguna venta. Se utiliza "IN"
231
232 SELECT e.nombre, e.apellido, e.nro_doc, e.tipo_doc
233 FROM empleados.EMPLEADO e
234 WHERE e.nro_doc IN (
235 SELECT k.nro_doc
236 FROM operaciones.VENTA K );
237
238 -- Productos que se vendieron en determinada fecha ( se indica 2025-08-01)
239 SELECT p.id_producto, p.detalle
240 FROM PRODUCTOS.PRODUCTO p
241 WHERE EXISTS (
242 SELECT 1
243 FROM operaciones.VENTA v
244 INNER JOIN operaciones.DETALLE_VENTA d
245 ON v.id_venta = d.id_venta
246 WHERE p.id_producto = d.id_producto

```

00 % 13 0

Resultados Mensajes

	nombre	apellido	nro_doc	tipo_doc
1	Juan	Perez	203040506	DNI
2	Ana	Gomez	203040507	DNI
3	Luis	Martinez	203040508	DNI
4	Franco	Gonzalez	203040509	DNI

```
238 -- Productos que se vendieron en determinada fecha ( se indica 2025-08-01)
239 SELECT p.id_producto, p.detalle
240 FROM PRODUCTOS.PRODUCTO p
241 WHERE EXISTS (
242     SELECT 1
243     FROM operaciones.VENTA v
244     INNER JOIN operaciones.DETALLE_VENTA d
245     ON v.id_venta = d.id_venta
246     WHERE p.id_producto = d.id_producto
247     AND CAST(v.hora_venta AS DATE) = '2025-08-01' -- <-- fecha deseada
248 );
249
250 -- Todas los empleados que vendieron para una fecha determinada
251
252 SELECT e.nombre, e.apellido, e.nro_doc, e.tipo_doc
253 FROM empleados.EMPLEADO e
254 WHERE e.nro_doc = ANY(
255     SELECT k.nro_doc
```

100 % 13 0

Resultados Mensajes

	id_producto	detalle
1	1	Cerveza rubia 500ml
2	2	Vino tinto 750ml
3	3	Gaseosa cola 2L

```
249
250 -- Todas los empleados que vendieron para una fecha determinada
251
252 SELECT e.nombre, e.apellido, e.nro_doc, e.tipo_doc
253 FROM empleados.EMPLEADO e
254 WHERE e.nro_doc = ANY(
255     SELECT k.nro_doc
256     FROM operaciones.VENTA k
257     WHERE CAST(k.hora_venta AS DATE) = '2025-08-01'
258 );
259
260 -- Todas los empleados que no vendieron en una fecha determinada
261
262 SELECT CONCAT(UPPER(e.apellido), ', ', e.nombre) AS nombre_completo, e.nro_doc,
263 FROM empleados.EMPLEADO e
264 WHERE e.nro_doc <> ALL(
```

100 % 13 0

Resultados Mensajes

	nombre	apellido	nro_doc	tipo_doc
1	Juan	Perez	203040506	DNI
2	Ana	Gomez	203040507	DNI
3	Luis	Martinez	203040508	DNI
4	Franco	Gonzalez	203040509	DNI

```

259
260 -- Todas los empleados que no vendieron en una fecha determinada
261
262 SELECT CONCAT(UPPER(e.apellido), ' ', e.nombre) AS nombre_completo, e.nro_doc, e.tipo_doc
263 FROM empleados.EMPLEADO e
264 WHERE e.nro_doc <> ALL(
265     SELECT k.nro_doc
266     FROM operaciones.VENTA k
267     WHERE CAST(k.hora_venta AS DATE) = '2025-08-01'
268 );
269
270 /*
271 3. Consultas con Funciones Comunes y de Agregados (Count(), Sum(), etc.
272 Se desarrollarán consultas que utilicen funciones comunes de SQL Server para resolver casos específicos del
273 sistema.

```

100 % 13 0

Resultados Mensajes

	nombre_completo	nro_doc	tipo_doc
1	GARCIA, Ezequiel	203040510	DNI
2	GIMENEZ, Leandro	203040511	DNI
3	GIMENEZ, Brian	203040512	DNI
4	SANCHEZ, Diego	203040513	DNI

```

270 /*
271 3. Consultas con Funciones Comunes y de Agregados (Count(), Sum(), etc.
272 Se desarrollarán consultas que utilicen funciones comunes de SQL Server para resolver casos específicos del
273 sistema.
274 • Funciones de texto.
275 • Funciones matemáticas.
276 • Funciones de fecha.
277 */
278
279 -- Cantidad de empleados utilizando count.
280
281 select COUNT(*) as Cant_empleados from empleados.EMPLEADO
282
283 -- Promedio de edad de empleados
284
285 select avg(a.edad) as Promedio_EDAD from empleados.EMPLEADO a

```

100 % 13 0

Resultados Mensajes

	Cant_empleados
1	8

```

279 -- Cantidad de empleados utilizando count.
280
281 select COUNT(*) as Cant_empleados from empleados.EMPLEADO
282
283 -- Promedio de edad de empleados
284
285 select avg(a.edad) as Promedio_EDAD from empleados.EMPLEADO a
286
287 -- Sumatoria total de las ventas
288
289 select sum(v.calc_venta) as Total_Ventas from operaciones.venta v
290
291 -- Funciones de texto Concat para concatenar nombre y apellido.
292
293 SELECT CONCAT(UPPER(e.apellido), ' ', e.nombre) AS nombre_completo, e.nro_doc, e.tipo_doc
294 FROM empleados.EMPLEADO e

```

100 % 13 0

Resultados Mensajes

	Promedio_EDAD
1	31

```

287 -- Sumatoria total de las ventas
288
289 select sum(v.calc_venta) as Total_Ventas from operaciones.venta v
290
291 -- Funciones de texto Concat para concatenar nombre y apellido.
292
293 SELECT CONCAT(UPPER(e.apellido), ' ', e.nombre) AS nombre_completo, e.nro_doc, e.tipo_
294 FROM empleados.EMPLEADO e
295
296 -- Modificacion de formato de fecha de nacimiento de empleado para una mejor visualizac
297
298 SELECT e.nombre, e.apellido, FORMAT(e.fecha_nac, 'dd/MM/yyyy') AS 'Día/Mes/Año' from em
299
300
301 /*
302 4. Funciones Definidas por el Usuario (UDF)
303 Se solicita la creación de al menos una función definida por el usuario que encapsule u

```

100 % 13 0

Resultados Mensajes

	Total_Ventas
1	12100

```

291 -- Funciones de texto Concat para concatenar nombre y apellido.
292
293 SELECT CONCAT(UPPER(e.apellido), ' ', e.nombre) AS nombre_completo, e.nro_doc, e.tipo_doc
294 FROM empleados.EMPLEADO e
295
296 -- Modificacion de formato de fecha de nacimiento de empleado para una mejor visualizacion.
297
298 SELECT e.nombre, e.apellido, FORMAT(e.fecha_nac, 'dd/MM/yyyy') AS 'Día/Mes/Año' from empleados.EMPLE
299
300
301 /*
302 4. Funciones Definidas por el Usuario (UDF)
303 Se solicita la creación de al menos una función definida por el usuario que encapsule una lógica esp
304 del sistema y pueda ser reutilizada en distintas consultas.
305 • Escalar
306 • De tabla en línea.
307 • De tabla multisentencia
308 */
309

```

00 % 13 0

Resultados Mensajes

	nombre_completo	nro_doc	tipo_doc
1	PEREZ, Juan	203040506	DNI
2	GOMEZ, Ana	203040507	DNI
3	MARTINEZ, Luis	203040508	DNI
4	GONZALEZ, Franco	203040509	DNI
5	GARCIA, Ezequiel	203040510	DNI
6	GIMENEZ, Leandro	203040511	DNI
7	GIMENEZ, Brian	203040512	DNI
8	SANCHEZ, Diego	203040513	DNI

```

296 -- Modificación de formato de fecha de nacimiento de empleado para una mejor visualización.
297
298 SELECT e.nombre, e.apellido, FORMAT(e.fecha_nac, 'dd/MM/yyyy') AS 'Día/Mes/Año' from empleados.EMPLEADO e
299
300
301 /*
302 4. Funciones Definidas por el Usuario (UDF)
303 Se solicita la creación de al menos una función definida por el usuario que encapsule una lógica específica
304 del sistema y pueda ser reutilizada en distintas consultas.
305 • Escalar
306 • De tabla en línea.
307 • De tabla multisentencia
308 */
309

```

100 % 13 0

Resultados Mensajes

	nombre	apellido	Día/Mes/Año
1	Juan	Perez	12/05/1985
2	Ana	Gomez	20/08/1990
3	Luis	Martinez	15/03/1988
4	Franco	Gonzalez	24/10/1998
5	Ezequiel	Garcia	24/10/1997
6	Leandro	Gimenez	24/10/1996
7	Brian	Gimenez	24/10/1996
8	Diego	Sanchez	24/10/1995

```

309
310 /*
311 4. Funciones Definidas por el Usuario (UDF)
312 Se solicita la creación de al menos una función definida por el usuario que encapsule una lógica específica
313 del sistema y pueda ser reutilizada en distintas consultas.
314 • Escalar
315 • De tabla en línea.
316 • De tabla multisentencia
317 */
318
319 --Funcion escalar donde obtenemos la cantidad de los productos con un stock menor al definido por el usuario
320
321 GO
322 CREATE FUNCTION dbo.CantProductosConMenorStock (@a SMALLINT)
323 RETURNS SMALLINT
324 AS
325 BEGIN
326     DECLARE @cant SMALLINT;
327
328     SELECT @cant = COUNT(*)
329     FROM productos.ALMACENA a
330     WHERE a.stock_actual < @a;
331
332     RETURN @cant;
333 END;
334 GO
335
336 select dbo.CantProductosConMenorStock (50) as cant_menor_stock;
337
338

```

132 % 7 0

Results Messages

cant_menor_stock
1 10

```

330
331 --Funcion de tabla en linea donde obtenemos la cantidad de los productos con un stock menor al definido por el usuario pero agrupado por kiosco
332
333 GO
334 CREATE FUNCTION dbo.fn_ProductosConMenorStock (@a SMALLINT)
335 RETURNS TABLE
336 AS
337 RETURN
338 (
339     SELECT
340         id_kiosko,
341         COUNT(*) AS CantidadProductos
342     FROM productos.ALMACENA
343     WHERE stock_actual < @a
344     GROUP BY id_kiosko
345 );
346 GO
347
348 SELECT *
349 FROM dbo.fn_ProductosConMenorStock(50);
350
351
352 --Funcion Multisentencia donde obtenemos la cantidad de productos con menor stock definido por el usuario y ademas obtenemos el producto del kiosco q
353

```

132 % 7 0

Results Messages

id_kiosko	CantidadProductos
1 1	3
2 2	4
3 3	3

```

352 --Funcion Multisentencia donde obtenemos la cantidad de productos con menor stock definido por el usuario y ademas obtenemos el producto del kiosco que menos stock tiene y la cantidad de ese stock
353
354
355 GO
356 CREATE FUNCTION dbo.fn_ProductosConMenorStockMultiTabla (@a SMALLINT)
357 RETURNS @Tabla TABLE
358 (
359     id_kiosko INT,
360     CantidadProductos INT,
361     id_producto_minimo INT,
362     StockMinimo SMALLINT
363 )
364 AS
365 BEGIN
366     INSERT INTO @Tabla
367     SELECT
368         id_kiosko,
369         COUNT(*) AS CantidadProductos,
370         -- subconsulta para traer el producto con menor stock
371         (
372             SELECT TOP 1 id_producto
373             FROM productos.ALMACENA x
374             WHERE x.id_kiosko = a.id_kiosko -- para que el logica sea para cada kiosko
375             AND x.stock_actual < @a
376             ORDER BY x.stock_actual ASC
377         ) AS id_producto_minimo,
378         MIN(stock_actual) AS StockMinimo
379     FROM productos.ALMACENA x
380     WHERE stock_actual < @a
381     GROUP BY id_kiosko;
382
383     RETURN;
384 END;
385 GO
386
387 SELECT *
388 FROM dbo.fn_ProductosConMenorStockMultiTabla (50);
389
390
391
392
393

```

id_kiosko	CantidadProductos	id_producto_minimo	StockMinimo
1	3	6	10
2	4	1	20
3	3	1	20

5. Operaciones de Actualización y Eliminación (UPDATE, DELETE)

Se incluirán ejemplos de:

- Actualización de datos en función de condiciones específicas.
- Eliminación de registros con control de integridad referencial.

*/

--Update estado actual del empleado que fue dado de baja

```
update [empleados].[EMPLEADO] set [estado] = 0 where [nro_doc] =203040507;
```

--Update fecha Hasta la que trabajo el empelado

```
UPDATE empleados.EMPLEADO
SET fecha_hasta = GETDATE()
WHERE nro_doc = 203040507;
```

--Delete de un empleado que se dio se alta erroneamente

```
INSERT INTO empleados.EMPLEADO
(nro_doc, tipo_doc, nombre, apellido, direccion, fecha_nac, id_kiosko, fecha_empieza, fecha_hasta, telefono)
VALUES
(203040514, 'DNI', 'JUAN', 'GIMENEZ', 'Calle X 450', '1995-10-24', 3, '2025-01-07', NULL, '3067-0854');
```

--Delete de una venta con detalle generada erroneamente, por la integridad referencial primero eliminamos el detalle de venta y luego la venta.
--Se debe validar el id de venta ya que la columna es identity tanto para borrar en tabla de ventas como en detalle de ventas

```
INSERT INTO operaciones.VENTA (modo_pago, hora_venta, id_turno, nro_doc, tipo_doc)
```

```
VALUES
('Tarjeta Debito', '2025-08-01 14:25', 1, 203040509, 'DNI');
GO
```

```
INSERT INTO operaciones.DETALLE_VENTA (nro_detalle, cantidad, precio_unitario, id_venta, id_producto)
```

```
VALUES
(1, 2, 50.00, 6, 1);
GO
```

```
delete from [operaciones].[DETALLE_VENTA] where id_venta = 6;
```

```
delete from [operaciones].[VENTA] where id_venta = 6;
```

```

/*
6.Consultas con Funciones de Ventana
Se utilizarán funciones como:
• OVER(PARTITION BY ...) para calcular agregados por grupo.
• COUNT(), SUM(), ROW_NUMBER(), RANK(), DENSE_RANK()
*/

-- Orden de ventas segun la fecha realizada por el empleado
SELECT
    [nro_doc],
    hora_venta,
    ROW_NUMBER() OVER (PARTITION BY [nro_doc] ORDER BY hora_venta) AS Orden
FROM operaciones.VENTA;

```

	nro_doc	hora_venta	Orden
1	203040506	2025-08-01 08:30:00.000	1
2	203040506	2025-08-01 10:30:00.000	2
3	203040506	2025-08-01 11:30:00.000	3
4	203040506	2025-08-01 12:30:00.000	4
5	203040506	2025-08-01 14:30:00.000	5
6	203040507	2025-08-01 14:25:00.000	1
7	203040508	2025-08-01 14:25:00.000	1
8	203040509	2025-08-01 14:25:00.000	1

--- Ranking de ventas por producto segun el monto total del mismo

```

SELECT
    k.id_kiosko,
    k.direccion,
    p.detalle AS nombre_producto,
    SUM(dv.subtotal) AS total_ventas_producto,
    ROW_NUMBER() OVER (PARTITION BY k.id_kiosko ORDER BY SUM(dv.subtotal) DESC) AS ranking_por_ventas
FROM operaciones.DETALLE_VENTA dv
INNER JOIN operaciones.VENTA v ON dv.id_venta = v.id_venta
INNER JOIN empleados.EMPLEADO e ON v.nro_doc = e.nro_doc AND v.tipo_doc = e.tipo_doc
INNER JOIN productos.KIOSKO k ON e.id_kiosko = k.id_kiosko
INNER JOIN productos.PRODUCTO p ON dv.id_producto = p.id_producto
GROUP BY k.id_kiosko, k.direccion, p.detalle
ORDER BY k.id_kiosko, total_ventas_producto DESC;

```

	id_kiosko	direccion	nombre_producto	total_ventas_producto	ranking_por_ventas
1	1	Av. Libertador 123	Vino tinto 750ml	4800.00	1
2	1	Av. Libertador 123	Cerveza rubia 500ml	4500.00	2
3	1	Av. Libertador 123	Gaseosa cola 2L	1800.00	3
4	2	Calle Falsa 456	Cerveza rubia 500ml	1000.00	1

```

-- Ranking de ventas por empleado por Dense Rank () y Rank () lo cual nos sirve para visualizar la diferencia ya que Dense_Rank no se saltea ningun ranking.
-- Agregamos registros para corroborar el funcionamiento de las funciones de ventana

INSERT INTO operaciones.VENTA (modo_pago, hora_venta, id_turno, nro_doc, tipo_doc)
VALUES
('Tarjeta Debito', '2025-08-01 14:25', 1, 203040507, 'DNI'),
('Tarjeta Debito', '2025-08-01 14:25', 1, 203040508, 'DNI'),
('Tarjeta Debito', '2025-08-01 14:25', 1, 203040509, 'DNI');
GO

INSERT INTO operaciones.DETALLE_VENTA (nro_detalle,cantidad, precio_unitario, id_venta, id_producto)
VALUES
(1,2, 1000, 7, 1),
(1,2, 1000, 8, 1),
(1,2, 500, 9, 1);
GO

```

```

SELECT
    e.nro_doc,
    e.nombre,
    e.apellido,
    SUM(dv.subtotal) AS total_ventas_empleado,
    DENSE_RANK() OVER (ORDER BY SUM(dv.subtotal) DESC) AS ranking_ventas
FROM empleados.EMPLEADO e
INNER JOIN operaciones.VENTA v ON e.nro_doc = v.nro_doc AND e.tipo_doc = v.tipo_doc
INNER JOIN operaciones.DETALLE_VENTA dv ON v.id_venta = dv.id_venta
GROUP BY e.nro_doc, e.nombre, e.apellido
ORDER BY ranking_ventas;

SELECT
    e.nro_doc,
    e.nombre,
    e.apellido,
    SUM(dv.subtotal) AS total_ventas_empleado,
    RANK() OVER (ORDER BY SUM(dv.subtotal) DESC) AS ranking_ventas
FROM empleados.EMPLEADO e
INNER JOIN operaciones.VENTA v ON e.nro_doc = v.nro_doc AND e.tipo_doc = v.tipo_doc
INNER JOIN operaciones.DETALLE_VENTA dv ON v.id_venta = dv.id_venta
GROUP BY e.nro_doc, e.nombre, e.apellido
ORDER BY ranking_ventas;

```

DENSE_RANK:

	nro_doc	nombre	apellido	total_ventas_empleado	ranking_ventas
1	203040506	Juan	Perez	7100.00	1
2	203040507	Ana	Gomez	2000.00	2
3	203040508	Luis	Martinez	2000.00	2
4	203040509	Franco	Gonzalez	1000.00	3

RANK:

	nro_doc	nombre	apellido	total_ventas_empleado	ranking_ventas
1	203040506	Juan	Perez	7100.00	1
2	203040507	Ana	Gomez	2000.00	2
3	203040508	Luis	Martinez	2000.00	2
4	203040509	Franco	Gonzalez	1000.00	4

7. Expresiones Comunes de Tabla (CTE)

Se desarrollarán consultas que utilicen WITH para definir CTEs, incluyendo:

- CTEs recursivas para jerarquías o estructuras anidadas.
- CTEs para simplificar subconsultas complejas.

*/

-- CTE no recursiva donde visualizamos el total de ventas por kiosko de los ultimos 90 dias de la fecha actual

```

WITH Ventas31d AS (
    SELECT v.id_venta, e.id_kiosko
    FROM operaciones.VENTA v
    JOIN empleados.EMPLEADO e
    ON e.nro_doc = v.nro_doc AND e.tipo_doc = v.tipo_doc
    WHERE v.hora_venta >= DATEADD(DAY, -90, GETDATE())
),
TotalesKiosko AS (
    SELECT vs.id_kiosko, SUM(d.subtotal) AS total_31d
    FROM Ventas31d vs
    JOIN operaciones.DETALLE_VENTA d ON d.id_venta = vs.id_venta
    GROUP BY vs.id_kiosko
)
SELECT id_kiosko, total_31d
FROM TotalesKiosko
ORDER BY total_31d DESC;

```

	id_kiosko	total_31d
1	1	11100.00
2	2	1000.00

```

-- CTE recursivo donde obtenemos el total total facturado por día de los últimos 90 días

DECLARE @desde DATE = DATEADD(DAY, -90, CAST(GETDATE() AS DATE));
DECLARE @hasta DATE = CAST(GETDATE() AS DATE);

;WITH Fechas AS (
    SELECT @desde AS fecha
    UNION ALL
    SELECT DATEADD(DAY, 1, fecha)
    FROM Fechas
    WHERE fecha < @hasta
)
SELECT
    f.fecha,
    SUM(d.subtotal) AS total_diario
FROM Fechas f
LEFT JOIN operaciones.VENTA v
    ON CAST(v.hora_venta AS DATE) = f.fecha
LEFT JOIN operaciones.DETALLE_VENTA d
    ON d.id_venta = v.id_venta
GROUP BY f.fecha
ORDER BY f.fecha
OPTION (MAXRECURSION 0); -- 0 = sin límite práctico, sigue hasta la fecha

```

	fecha	total_diario
52	2025-07-24	NULL
53	2025-07-25	NULL
54	2025-07-26	NULL
55	2025-07-27	NULL
56	2025-07-28	NULL
57	2025-07-29	NULL
58	2025-07-30	NULL
59	2025-07-31	NULL
60	2025-08-01	12100.00
61	2025-08-02	NULL
62	2025-08-03	NULL
63	2025-08-04	NULL
64	2025-08-05	NULL

8-

```

8.Transformaciones con PIVOT y UNPIVOT
Se incluirán ejemplos de:
• Transformación de datos verticales a horizontales (PIVOT).
• Reversión de estructuras (UNPIVOT) para análisis dinámico.
*/

-- PIVOT sobre la tabla de ventas para obtener el total facturado por mes de cada empleado
SELECT
    nro_doc,
    [1] AS Enero,
    [2] AS Febrero,
    [3] AS Marzo,
    [4] AS Abril,
    [5] AS Mayo,
    [6] AS Junio,
    [7] AS Julio,
    [8] AS Agosto,
    [9] AS Septiembre,
    [10] AS Octubre,
    [11] AS Noviembre,
    [12] AS Diciembre
FROM (
    SELECT
        -- PIVOT
        a.nro_doc,
        MONTH(b.hora_venta) AS Mes,
        b.calc_venta
    FROM empleados.EMPLEADO a
    LEFT JOIN operaciones.VENTA b
        ON a.nro_doc = b.nro_doc
) AS SourceTable
PIVOT
(
    SUM(calc_venta)
    FOR Mes IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])
) AS PivotTable
ORDER BY nro_doc;

```

	nro_doc	Enero	Febrero	Marzo	Abril	Mayo	Junio	Julio	Agosto	Septiembre	Octubre	Noviembre	Diciembre
1	203040506	NULL	NULL	NULL	NULL	NULL	NULL	NULL	7100	NULL	NULL	NULL	NULL
2	203040507	NULL	NULL	NULL	NULL	NULL	NULL	NULL	2000	NULL	NULL	NULL	NULL
3	203040508	NULL	NULL	NULL	NULL	NULL	NULL	NULL	2000	NULL	NULL	NULL	NULL
4	203040509	NULL	NULL	NULL	NULL	NULL	NULL	NULL	1000	NULL	NULL	NULL	NULL
5	203040510	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
6	203040511	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
7	203040512	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
8	203040513	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

-- Unpivot donde creamos una vista estandar sobre el total facturado por mes y pasamos las columnas a filas.

CREATE VIEW TOTAL_FACTURADO_POR_MES AS

```

SELECT
    nro_doc,
    [1] AS Enero,
    [2] AS Febrero,
    [3] AS Marzo,
    [4] AS Abril,
    [5] AS Mayo,
    [6] AS Junio,
    [7] AS Julio,
    [8] AS Agosto,
    [9] AS Septiembre,
    [10] AS Octubre,
    [11] AS Noviembre,
    [12] AS Diciembre
FROM (
    SELECT
        a.nro_doc,
        MONTH(b.hora_venta) AS Mes,
        b.calc_venta
    FROM empleados.EMPLEADO a
    LEFT JOIN operaciones.VENTA b
        ON a.nro_doc = b.nro_doc
) AS SourceTable
PIVOT
(
    SUM(calc_venta)
    FOR Mes IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])
) AS PivotTable;

SELECT
    nro_doc,
    Mes,
    Importe,
    CASE Mes
        WHEN 'Enero' THEN 1
        WHEN 'Febrero' THEN 2
        WHEN 'Marzo' THEN 3
        WHEN 'Abril' THEN 4
        WHEN 'Mayo' THEN 5
        WHEN 'Junio' THEN 6
        WHEN 'Julio' THEN 7
        WHEN 'Agosto' THEN 8
        WHEN 'Septiembre' THEN 9
        WHEN 'Octubre' THEN 10
        WHEN 'Noviembre' THEN 11
        WHEN 'Diciembre' THEN 12
    END AS MesNum
FROM dbo.TOTAL_FACTURADO_POR_MES
UNPIVOT (
    Importe FOR Mes IN (
        Enero, Febrero, Marzo, Abril, Mayo, Junio,
        Julio, Agosto, Septiembre, Octubre, Noviembre, Diciembre
    )
) AS uno;
```

	nro_doc	Mes	Importe	MesNum
1	203040506	Agosto	7100	8
2	203040507	Agosto	2000	8
3	203040508	Agosto	2000	8
4	203040509	Agosto	1000	8

9-

9. Vistas

Se solicita la creación de los tipos de vistas vistas, que reflejen distintos aspectos del sistema implementado. Cada vista debe estar acompañada de una breve descripción de su propósito y utilidad.

*/

-- Vista Estandar donde guardamos los empleados que pertenecen al KISKO ID 1, nos sirve para no escribir la consula multiples veces.

```
create view EMPLEADOS_EN_KISKO_1 AS
```

```
SELECT * FROM [empleados].[EMPLEADO] WHERE [id_kiosko] = 1;
```

Columnas
nro_doc (numeric(9,0), No NULL)
tipo_doc (varchar(10), No NULL)
nombre (varchar(50), No NULL)
apellido (varchar(50), No NULL)
direccion (varchar(100), NULL)
estado (bit, No NULL)
fecha_nac (date, NULL)
edad (int, NULL)
id_kiosko (int, No NULL)
fecha_hasta (date, NULL)
fecha_empieza (date, No NULL)
telefono (varchar(20), NULL)
antiguedad_anio (int, NULL)

```
CREATE VIEW empleados.v_EmpleadoNacionalidad
WITH SCHEMABINDING
AS
```

```
SELECT
    e.nro_doc,
    e.tipo_doc,
    e.nombre,
    e.apellido,
    e.estado,
    e.id_kiosko,
    e.fecha_empieza,
    e.telefono,
    n.nacionalidad
FROM empleados.EMPLEADO AS e
JOIN empleados.NACIONALIDAD AS n
    ON e.nro_doc = n.nro_doc
    AND e.tipo_doc = n.tipo_doc;
GO
```

-- Indice clustered unique

```
CREATE UNIQUE CLUSTERED INDEX IX_v_EmpleadoNac
ON empleados.v_EmpleadoNacionalidad (nro_doc, tipo_doc, nacionalidad);
GO
```

Índices
IX_v_EmpleadoNac (clúster)

Nombre de vista:

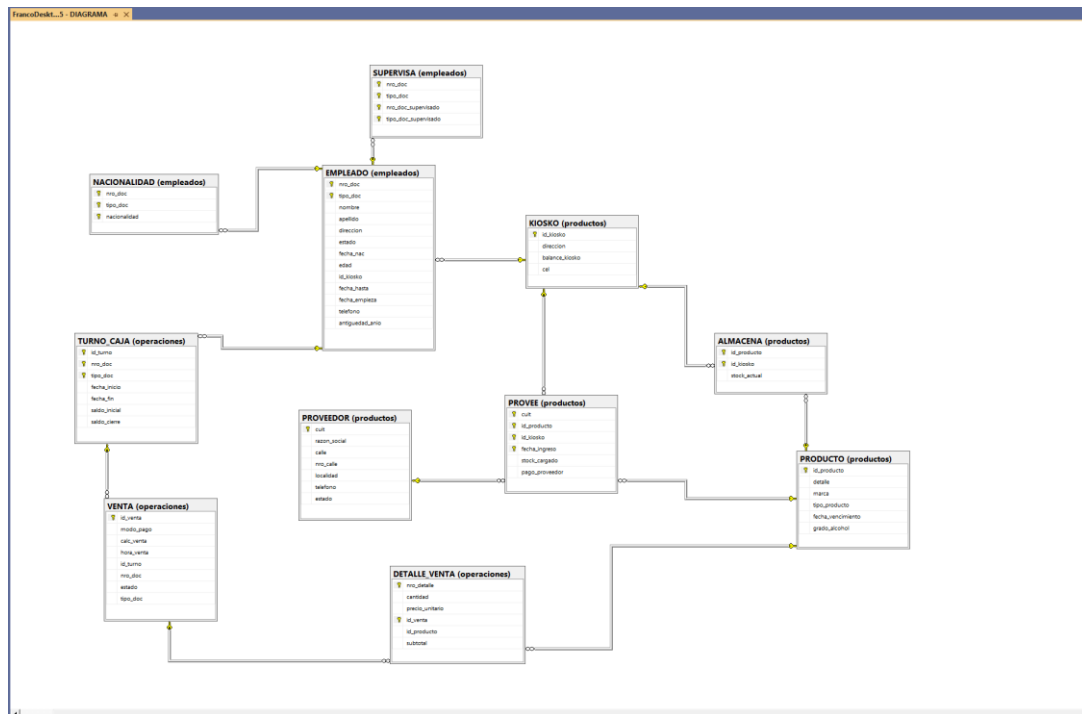
Nombre de índice:

Tipo de índice:

☒ Único

Clave de índice columnas

Nombre	Criterio de ordenación	Tipo de datos	Tamaño	Identidad	Permitir valores NULL	Agregar...
nro_doc	Ascendente	numeric(9,0)	5	No	No	Quitar
tipo_doc	Ascendente	varchar(10)	10	No	No	Subir
nacionalidad	Ascendente	varchar(30)	30	No	No	Bajar



Parte 4- Operaciones con T-SQL y migración de datos.

Aplicar técnicas avanzadas de manipulación de datos (DML), automatización mediante triggers, procesamiento iterativo con cursores, y operaciones de integración de datos desde fuentes externas (JSON y CSV), utilizando SQL Server. El objetivo es demostrar la capacidad de extender el modelo relacional con lógica programática y procesos de actualización controlada

```
/*1. Procedimientos almacenados (Stored Procedures -SP-)
Se deberán desarrollar al menos tres procedimientos almacenados que aborden
distintas operaciones sobre
la base de datos:
a) SP de consulta parametrizada
Recibe uno o mas parametros de entrada.
Devuelve un conjunto de datos filtrado desde una tabla del sistema.
Puede incluir un parametro de salida o un mensaje informativo que indique el
estado de la operacion.*/

--1A
--Busqueda simple
USE GRUPO5;
GO

CREATE OR ALTER PROCEDURE empleados.SP_EmpleadosBuscar_Simple
    @id_kiosko      INT          = NULL,    -- filtra por kiosko
    @estado         VARCHAR(20)  = NULL,    -- 'ACTIVO' / 'BAJA'
    @apellido_like  VARCHAR(50)  = NULL,    -- busca coincidencia parcial en el
apellido
    @filas          INT          OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        e.nro_doc,
        e.tipo_doc,
        e.nombre,
        e.apellido,
        e.estado,
        e.id_kiosko,
        e.fecha_empieza,
        e.fecha_hasta
    FROM empleados.EMPLEADO e
    WHERE (@id_kiosko IS NULL OR e.id_kiosko = @id_kiosko)
        AND (@estado IS NULL OR e.estado = @estado)
        AND (@apellido_like IS NULL OR e.apellido LIKE '%' + @apellido_like + '%')
    ORDER BY e.apellido, e.nombre;

    SET @filas = @@ROWCOUNT;
    RAISERROR('Consulta OK. Filas: %d', 10, 1, @filas) WITH NOWAIT;
```

```
END
GO

-- Ejemplo: buscar por apellido
DECLARE @m INT;
EXEC empleados.SP_EmpleadosBuscar_Simple
    @apellido_like = 'Gomez',
    @filas = @m OUTPUT;
SELECT @m AS filas;
```

```
/*
b) SP de insercion de datos
Inserta registros en una tabla principal del sistema.
Debe incluir validaciones previas, como verificar que los valores ingresados sean
correctos y que los campos obligatorios tengan contenido.
Devuelve un mensaje de confirmacion o el identificador del registro insertado como
parametro de salida.*/

--1B
--procedimiento para crear un producto nuevo, con chequeos mínimos, y entrega el
identificador del registro que queda grabado.
--Si faltan los campos obligatorios, no inserta y avisa por error.

GO
CREATE OR ALTER PROCEDURE productos.SP_ProductoInsertar_Min
    @detalle NVARCHAR(200),
    @marca VARCHAR(50),
    @tipo_producto VARCHAR(20) = NULL,
    @id_nuevo INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validaciones m nimas
    IF (@detalle IS NULL OR LTRIM(RTRIM(@detalle)) = '')
    BEGIN
        RAISERROR('detalle es obligatorio.', 16, 1);
        RETURN;
    END

    IF (@marca IS NULL OR LTRIM(RTRIM(@marca)) = '')
    BEGIN
        RAISERROR('marca es obligatoria.', 16, 1);
        RETURN;
    END
END
```

```
-- Inserción
INSERT INTO productos.PRODUCTO (detalle, marca, tipo_producto)
VALUES (@detalle, @marca, @tipo_producto);

-- ID generado
SET @id_nuevo = SCOPE_IDENTITY();

-- Mensaje de confirmación
RAISERROR('Insert OK. id_producto=%d', 10, 1, @id_nuevo) WITH NOWAIT;
END
GO

--Prueba
DECLARE @id INT;
EXEC productos.SP_ProductoInsertar_Min
    @detalle = N'Agua 500ml',
    @marca = 'Manaos',
    @tipo_producto = 'BEBIDA',
    @id_nuevo = @id OUTPUT;
SELECT @id AS id_nuevo;

/*
c) SP de eliminación controlada
Elimina registros de una tabla según condiciones específicas definidas por el
usuario.
Debe incluir manejo de errores mediante bloques TRY...CATCH, y controlar la
integridad referencial para evitar conflictos con claves foráneas.
Devuelve un mensaje que indique si la operación fue exitosa o el motivo por el
cual fue rechazada.*/

--1.3
-- SP de eliminación controlada (con chequeos y TRY...CATCH) de un producto
GO
CREATE OR ALTER PROCEDURE productos.SP_ProductoEliminar_Controlado
    @id_producto INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM productos.PRODUCTO WHERE
id_producto=@id_producto)
            THROW 80001, 'Producto inexistente.', 1;

        IF EXISTS (SELECT 1 FROM operaciones.DETALLE_VENTA WHERE
id_producto=@id_producto)
            THROW 80002, 'No se puede eliminar: usado en DETALLE_VENTA.', 1;
```

```
IF EXISTS (SELECT 1 FROM productos.ALMACENA WHERE
id_producto=@id_producto)
    THROW 80003, 'No se puede eliminar: existe stock en ALMACENA.', 1;

IF EXISTS (SELECT 1 FROM productos.PROVEE WHERE id_producto=@id_producto)
    THROW 80004, 'No se puede eliminar: referenciado en PROVEE.', 1;

DELETE FROM productos.PRODUCTO WHERE id_producto=@id_producto;
RAISERROR('Producto eliminado (id=%d).', 10, 1, @id_producto) WITH NOWAIT;
END TRY
BEGIN CATCH
    DECLARE @msg NVARCHAR(4000)=ERROR_MESSAGE();
    RAISERROR('Eliminaci3n rechazada: %s', 16, 1, @msg);
END CATCH
END
GO

-- Tomo el id más nuevo de ese producto y lo elimino
DECLARE @id INT;

SELECT @id = MAX(id_producto)      -- el duplicado más reciente
FROM productos.PRODUCTO
WHERE detalle = N'Agua 500ml'
    AND marca   = 'Manaos'
    AND tipo_producto = 'BEBIDA';

-- se comenta para no borrar
--EXEC productos.SP_ProductoEliminar_Controlado @id_producto=@id;

/*
2. Triggers
Se deber3n implementar al menos dos triggers que automaticen acciones sobre la
base de datos:
a) Trigger AFTER INSERT
Este trigger se ejecutar3 automáticamente después de una inserci3n en una tabla
principal. Su objetivo
ser3 registrar los datos insertados en una tabla de auditoría, permitiendo llevar
un control de los
cambios realizados.*/

/* 2-a creaci3n de trigger para actualizar stock_actual en almacena */

-- Trigger AFTER INSERT en PROVEE. Cuando se realiza un ingreso de un proveedor
actualizo el stock en la tabla almacena en el kiosko ingresado.

IF OBJECT_ID('trg_insertar_stock', 'TR') IS NOT NULL
    DROP TRIGGER trg_insertar_stock;
GO
```

```
CREATE TRIGGER trg_insertar_stock
ON productos.PROVEE
AFTER INSERT
AS
BEGIN
    UPDATE a
    SET a.stock_actual = a.stock_actual + i.stock_cargado
    FROM productos.ALMACENA a
    INNER JOIN inserted i
        ON a.id_producto = i.id_producto
        AND a.id_kiosko = i.id_kiosko;
END
GO

-- Trigger AFTER DELETE en PROVEE. Es el inverso del trigger anterior. Por si se
-- ingresa un valor erroneo.
IF OBJECT_ID('trg_borrar_stock', 'TR') IS NOT NULL
    DROP TRIGGER trg_borrar_stock;
GO
CREATE TRIGGER trg_borrar_stock
ON productos.PROVEE
AFTER DELETE
AS
BEGIN
    UPDATE a
    SET a.stock_actual = a.stock_actual - d.stock_cargado
    FROM productos.ALMACENA a
    INNER JOIN deleted d
        ON a.id_producto = d.id_producto
        AND a.id_kiosko = d.id_kiosko;
END
GO

-- Trigger AFTER UPDATE en PROVEE
IF OBJECT_ID('trg_actualizar_stock', 'TR') IS NOT NULL
    DROP TRIGGER trg_actualizar_stock;
GO
CREATE TRIGGER trg_actualizar_stock
ON productos.PROVEE
AFTER UPDATE
AS
BEGIN
    UPDATE a
    SET a.stock_actual = a.stock_actual - d.stock_cargado + i.stock_cargado
    FROM productos.ALMACENA a
    INNER JOIN inserted i
        ON a.id_producto = i.id_producto
        AND a.id_kiosko = i.id_kiosko
```

```
        INNER JOIN deleted d
            ON a.id_producto = d.id_producto
            AND a.id_kiosko = d.id_kiosko;
END
GO

--TRIGER PARA CALCULAR VENTA
-- Trigger AFTER INSERT en detalle_venta - Trigger para calcular el total de la
venta sumando los subtotales de los detalles. Luego de ingresar un detalle de
venta
-- Se suma el valor del subtotal en calc_venta en la tabla venta.
IF OBJECT_ID('trg_insertar_subtotal', 'TR') IS NOT NULL
    DROP TRIGGER trg_insertar_subtotal;
GO
CREATE TRIGGER trg_insertar_subtotal
ON operaciones.detalle_venta
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE v
    SET v.calc_venta = ISNULL(v.calc_venta, 0) + t.suma_subtotal
    FROM operaciones.VENTA v
    INNER JOIN (
        SELECT id_venta, SUM(subtotal) AS suma_subtotal
        FROM inserted
        GROUP BY id_venta
    ) t ON v.id_venta = t.id_venta;
END
GO

-- Trigger para AFTER DELETE en detalle_venta. Inverso del Trigger anterior para
corregir cargas erróneas.
IF OBJECT_ID('trg_borrar_subtotal', 'TR') IS NOT NULL
    DROP TRIGGER trg_borrar_subtotal;
GO
CREATE TRIGGER trg_borrar_subtotal
ON operaciones.detalle_venta
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE v
    SET v.calc_venta = ISNULL(v.calc_venta, 0) - t.suma_subtotal
    FROM operaciones.VENTA v
    INNER JOIN (
        SELECT id_venta, SUM(subtotal) AS suma_subtotal
```

```
        FROM deleted
        GROUP BY id_venta
    ) t ON v.id_venta = t.id_venta;
END
GO

-- Trigger AFTER INSERT en PROVEE: restar el pago a proveedor. Cuando se ingresa
stock de un producto de un proveedor en un kiosko se resta del balance del kiosko
correspondiente.
IF OBJECT_ID('trg_pago_proveedor_insert', 'TR') IS NOT NULL
    DROP TRIGGER trg_pago_proveedor_insert;
GO
CREATE TRIGGER trg_pago_proveedor_insert
ON productos.PROVEE
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE k
    SET k.balance_kiosko = ISNULL(k.balance_kiosko, 0) - t.total_pago
    FROM productos.KIOSKO k
    INNER JOIN (
        SELECT p.id_kiosko, SUM(ISNULL(p.pago_proveedor,0)) AS total_pago
        FROM inserted p
        GROUP BY p.id_kiosko
    ) t ON k.id_kiosko = t.id_kiosko;
END;
GO

-- Trigger AFTER DELETE en productos.PROVEE (restituye pagos borrados). Inverso
del trigger anterior.
IF OBJECT_ID('trg_pago_proveedor_delete', 'TR') IS NOT NULL
    DROP TRIGGER trg_pago_proveedor_delete;
GO
CREATE TRIGGER trg_pago_proveedor_delete
ON productos.PROVEE
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE k
    SET k.balance_kiosko = ISNULL(k.balance_kiosko, 0) + t.total_pago
    FROM productos.KIOSKO k
    INNER JOIN (
        SELECT d.id_kiosko, SUM(ISNULL(d.pago_proveedor,0)) AS total_pago
        FROM deleted d
```

```

        GROUP BY d.id_kiosko
    ) t ON k.id_kiosko = t.id_kiosko;
END;
GO

/*
b) Trigger INSTEAD OF UPDATE
Este trigger se utilizará para interceptar actualizaciones en una tabla
especifica. Antes de permitir la modificación, deberá validar ciertas condiciones
(por ejemplo, que el estado del registro lo permita). Si no se cumplen, se podrá
cancelar la operación o registrar un mensaje de advertencia.*/

--
CREATE TRIGGER trg_InsteadOfUpdate_Empleado
ON empleados.EMPLEADO
INSTEAD OF UPDATE
AS
BEGIN
    -- Update controlado
    UPDATE e
    SET
        e.nombre      = ISNULL(i.nombre, e.nombre),
        e.apellido     = ISNULL(i.apellido, e.apellido),
        e.direccion    = ISNULL(i.direccion, e.direccion),
        e.estado       = i.estado,
        e.fecha_nac    = i.fecha_nac,
        e.id_kiosko    = i.id_kiosko,
        e.fecha_empieza = i.fecha_empieza,
        e.telefono     = i.telefono,
        e.fecha_hasta  = CASE
            WHEN i.estado = 0 AND e.fecha_hasta IS NULL
            THEN GETDATE() -- si lo ponen inactivo, fecha
actual
                        ELSE e.fecha_hasta -- si no, conserva el valor
            END
    FROM empleados.EMPLEADO e
    INNER JOIN inserted i
        ON e.nro_doc = i.nro_doc
        AND e.tipo_doc = i.tipo_doc;
END;
GO

-- Pasa al empleado a inactivo y le pone fecha_hasta = hoy
UPDATE empleados.EMPLEADO
SET estado = 0
WHERE nro_doc = 203040508 AND tipo_doc = 'dni';

```

```
-----  
  
/*  
3. Cursor dentro de un Stored Procedure  
Se debera desarrollar un Stored Procedure que incluya el uso de un cursor para  
recorrer una tabla del sistema y realizar una operación por cada fila.  
El procedimiento debe recibir al menos un parámetro de entrada, que será utilizado  
para filtrar o condicionar la consulta.  
Por cada fila recorrida, se deberá realizar un cálculo simple (por ejemplo, sumar  
valores, calcular porcentajes, etc.) y mostrar el resultado por pantalla  
utilizando PRINT o una variable de salida.  
El cursor debe estar correctamente declarado, abierto, utilizado y cerrado,  
siguiendo las buenas prácticas de SQL Server.  
*/  
  
-- Se genera un storeProcedure que incluye un Cursor. Su funcionalidad nos permite  
obtener el promedio de edad de empleados activos de un kiosko indicado.  
GO  
CREATE PROCEDURE empleados.CalcularPromedioEdad  
    @idKiosko INT    -- parámetro de entrada para filtrar empleados de un kiosko  
AS  
BEGIN  
    SET NOCOUNT ON;  
    DECLARE  
        @edad INT,  
        @sumaEdad INT = 0,  
        @contador INT = 0,  
        @promedio DECIMAL(10,2);  
    -- Declaración del cursor  
    DECLARE curEmpleado CURSOR FOR  
        SELECT edad  
        FROM empleados.EMPLEADO  
        WHERE id_kiosko = @idKiosko AND estado = 1; -- solo empleados activos  
    -- Abrir el cursor  
    OPEN curEmpleado;  
    -- Leer la primera fila  
    FETCH NEXT FROM curEmpleado INTO @edad;  
    -- Recorrer mientras haya filas  
    WHILE @@FETCH_STATUS = 0  
    BEGIN  
        SET @sumaEdad = @sumaEdad + @edad; -- acumulamos edades  
        SET @contador = @contador + 1;  
        -- Traer la siguiente fila  
        FETCH NEXT FROM curEmpleado INTO @edad;  
    END;  
    -- Cerrar y liberar cursor  
    CLOSE curEmpleado;  
    DEALLOCATE curEmpleado;
```

```

-- Calcular promedio si hay empleados
IF @contador > 0
BEGIN
    SET @promedio = CAST(@sumaEdad AS DECIMAL(10,2)) / @contador;
    PRINT 'El promedio de edad de los empleados del kiosko '
        + CAST(@idKiosko AS VARCHAR(10))
        + ' es ' + CAST(@promedio AS VARCHAR(10));
END
ELSE
BEGIN
    PRINT 'No hay empleados activos para el kiosko ' + CAST(@idKiosko AS
VARCHAR(10));
END
END;
GO

```

--Uso del mismo. Indicando el kiosko 1.

```
EXEC empleados.CalcularPromedioEdad @idKiosko = 1;
```

/*

4. Importación de Datos desde JSON

Se deberá realizar la carga de datos desde un archivo JSON real, previamente creado o obtenido desde una fuente externa confiable (por ejemplo, una URL oficial del Gobierno de la Provincia de Buenos Aires).

Pasos sugeridos:

Crear una tabla en la base de datos que reciba los datos del archivo JSON.

Cargar los datos desde un archivo para leer el contenido del archivo y transformarlo en registros tabulares.

Insertar los datos obtenidos en la tabla correspondiente.

Mostrar los registros insertados como resultado de la operación.

*/

--Tabla receptora

```

IF OBJECT_ID('productos.PRODUCTO') IS NULL
CREATE TABLE productos.PRODUCTO(
    detalle NVARCHAR(200) NOT NULL,
    marca   VARCHAR(50)   NOT NULL,
    tipo_producto VARCHAR(20) NULL,
    fecha_vencimiento DATE NULL,
    grado_alcohol DECIMAL(4,2) NULL
);

```

--recordar editar la ubicacion de la carpeta donde tengo el archivo json

```
DECLARE @json NVARCHAR(MAX);
```

```
SELECT @json = BulkColumn
```

```
FROM OPENROWSET(BULK
'C:\Users\franc\OneDrive\Educacion\Universidad\Maestrias\Posgrado en Ciencia de
Datos\Segundo Cuatrimestre\Topicos de base de datos\Clase 5 - 30-
8\TP4\productos.json', SINGLE_CLOB) AS j;

INSERT INTO productos.PRODUCTO(detalle, marca, tipo_producto, fecha_vencimiento,
grado_alcohol)
SELECT detalle, marca, tipo_producto, fecha_vencimiento, grado_alcohol
FROM OPENJSON(@json)
WITH(
    detalle NVARCHAR(200) '$.detalle',
    marca VARCHAR(50) '$.marca',
    tipo_producto VARCHAR(20) '$.tipo_producto',
    fecha_vencimiento DATE '$.fecha_vencimiento',
    grado_alcohol DECIMAL(4,2) '$.grado_alcohol'
);

SELECT * FROM productos.PRODUCTO; -- mostrar insertados

/*

5. Actualización de Datos desde CSV
Se debera realizar la actualización de una tabla del sistema utilizando datos
provenientes de un archivo en formato CSV, previamente guardado en el sistema de
archivos del servidor SQL o en una ruta accesible.
Importar el archivo CSV a una tabla temporal en la base de datos.
Comparar los registros de la tabla temporal con los de la tabla principal,
utilizando condiciones como claves primarias o campos únicos.
Ejecutar la sentencia UPDATE solo sobre los registros que no están previamente
cargados o que presenten diferencias relevantes.
Mostrar los registros actualizados como resultado de la operación.
Este ejercicio tiene como objetivo practicar la integración de datos externos en
el sistema, aplicar técnicas de control de duplicados y realizar actualizaciones
condicionales de forma segura y eficiente.
*/

-- Limpieza previa por si ya existe
IF OBJECT_ID('tempdb..#TMP_PRODUCTOS_READ') IS NOT NULL
    DROP TABLE #TMP_PRODUCTOS_READ;

CREATE TABLE #TMP_PRODUCTOS_READ(
    detalle NVARCHAR(200) NOT NULL,
    marca VARCHAR(50) NOT NULL,
    tipo_producto VARCHAR(20) NULL,
    fecha_vencimiento NVARCHAR(20) NULL,
    grado_alcohol NVARCHAR(20) NULL
);
```

```
BULK INSERT #TMP_PRODUCTOS_READ
FROM 'C:\Users\franc\OneDrive\Educacion\Universidad\Maestrias\Posgrado en Ciencia
de Datos\Segundo Cuatrimestre\Topicos de base de datos\Clase 5 - 30-
8\TP4\productos.csv'
WITH (
    FIRSTROW = 2,          -- Salta cabecera si tiene
    FIELDTERMINATOR = ',', -- Separador CSV
    ROWTERMINATOR = '\n',  -- Fin de línea
    CODEPAGE = '65001',    -- UTF-8
    TABLOCK
);

-- Se convierte los tipos

IF OBJECT_ID('tempdb..#TMP_PRODUCTOS') IS NOT NULL
    DROP TABLE #TMP_PRODUCTOS;

CREATE TABLE #TMP_PRODUCTOS(
    detalle NVARCHAR(200) NOT NULL,
    marca   VARCHAR(50)   NOT NULL,
    tipo_producto VARCHAR(20) NULL,
    fecha_vencimiento DATE NULL,
    grado_alcohol DECIMAL(4,2) NULL
);

INSERT INTO #TMP_PRODUCTOS (detalle, marca, tipo_producto, fecha_vencimiento,
grado_alcohol)
SELECT
    detalle,
    marca,
    tipo_producto,
    TRY_CONVERT(DATE, NULLIF(LTRIM(RTRIM(fecha_vencimiento)), ''), 120), --
convierte '' a NULL
    TRY_CONVERT(DECIMAL(4,2), NULLIF(LTRIM(RTRIM(grado_alcohol)), '')) --
convierte '' a NULL, luego a DECIMAL
FROM #TMP_PRODUCTOS_READ;

MERGE productos.PRODUCTO AS target
USING #TMP_PRODUCTOS AS source
ON target.detalle = source.detalle
AND target.marca = source.marca
WHEN MATCHED AND (
    target.tipo_producto <> source.tipo_producto
OR target.fecha_vencimiento <> source.fecha_vencimiento
OR target.grado_alcohol <> source.grado_alcohol
)
```

```
    THEN UPDATE SET
        target.tipo_producto      = source.tipo_producto,
        target.fecha_vencimiento = source.fecha_vencimiento,
        target.grado_alcohol      = source.grado_alcohol
WHEN NOT MATCHED BY TARGET
    THEN INSERT (detalle, marca, tipo_producto, fecha_vencimiento, grado_alcohol)
        VALUES (source.detalle, source.marca, source.tipo_producto,
source.fecha_vencimiento, source.grado_alcohol);

SELECT p.* FROM productos.PRODUCTO p;
```